Advanced · LLM customisation strategy · Highlights · IBM Technology

Does Fine-Tuning Still Earn Its Place?

Work out where custom weights still beat a frontier model plus a good context stack, and where they no longer do.

For engineers and leads deciding whether to invest in custom model training · 26 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

Watch the source video · Read the full lesson · Read the condensed version

---

What matters

- ✓ Fine-tuning continues a base model's training on a focused dataset, adjusting its weights toward a narrow domain.
- ✓ A fine-tuned model is a fixed artefact competing against frontier models that keep improving — a moving target.
- ✓ Large context windows removed the main reason to bake documents into weights: you can now just supply them.
- ✓ Reasoning models shift capability to inference time, so 'how hard it thinks now' rivals 'how it was trained months ago'.
- ✓ RAG, context engineering and agent skills customise behaviour without modifying any weights.
- ✓ Agent skills package procedural knowledge as loadable files, so any general model can perform a specialised procedure.
- ✓ Most production fine-tuning today is LoRA or a related parameter-efficient method that trains a small adapter over frozen weights.
- ✓ Fine-tuning still wins for hard latency constraints, distillation into smaller models, and reinforcement fine-tuning against a programmatic grader.

What it covers

01 **The 2023 case, and why it was convincing** `0:00`
A custom legal model beat the leading frontier model in blind tests, decisively. The conclusion drawn was that domain work means fine-tuning.

02 **What fine-tuning does to a model** `0:41`
Take a base model trained on a broad corpus and continue training it on a focused dataset, producing a new model with adjusted weights.

03 **The 2025 rematch** `2:10`
The same company built a legal benchmark and found seven general-purpose models, with no legal fine-tuning at all, had surpassed their custom one.

04 **Why general models caught up** `3:35`
Three changes: context windows grew enormously, reasoning moved to inference time, and inference got cheaper.

05 **Customisation without touching the weights** `5:41`
RAG supplies the knowledge, and context engineering assembles everything the model needs into a deliberately constructed prompt.

06 **Agent skills: packaging procedural knowledge** `6:27`
Folders of files that package how to do something and which tools to use, loaded on demand when the model meets a matching task.

07 **LoRA and parameter-efficient fine-tuning** `7:11`
Train a small adapter over frozen base weights. Most of what is called fine-tuning in production today is some flavour of this.

08 **Where fine-tuning still wins** `7:52`
Three specific bottlenecks the no-weights stack cannot solve: hard latency limits, distillation into smaller models, and training against a programmatic grader.

09 **The decision order** `10:01`
Base model, then prompt and context engineering, then RAG, then skills — and fine-tuning only for a bottleneck the rest cannot clear.

Glossary

Base model — The off-the-shelf pre-trained model, from a frontier lab or open source, before any customisation.

Fine-tuning — Continuing a base model's training on a focused dataset so the domain behaviour ends up in the weights.

Moving target — The problem that a fixed custom model competes against frontier models that keep improving, so its lead decays without anything changing.

Context window — How much text a model can process in one call. Growth from thousands to millions of tokens removed much of the need to bake knowledge into weights.

Reasoning model — A model that performs extended step-by-step thinking at inference time, shifting capability from training to the moment of the question.

Context engineering — Deliberately assembling the prompt from parts — system role, retrieved data, examples, format rules — as a versioned, testable artefact.

Agent skill — A folder of files packaging procedural knowledge and the tools to apply it, loaded on demand by any capable model.

LoRA — Low-rank adaptation: training two small matrices whose product approximates a weight update, while the base weights stay frozen.

PEFT — Parameter-efficient fine-tuning: the family of methods, including LoRA, that adapt a model by training a small fraction of parameters.

Adapter — The small trained artefact a PEFT method produces. Swappable, stackable against one base model, and removable.

Rank (r) — The bottleneck width of a LoRA adapter, controlling its capacity and its parameter count.

Distillation — Using a large teacher model to generate high-quality outputs, then fine-tuning a smaller student model on them.

Reinforcement fine-tuning (RFT) — Training against a grader rather than fixed answers: the model samples candidates, the grader scores them, and high scores are reinforced.

Grader — A programmatic scoring function used in RFT. Requires a definitive checkable answer, which is what limits where RFT applies.